

bbtkv3: An Open-Source Suite for Timing Measurement and Analysis with the Black Box ToolKit v3

Christophe Pallier
Cognitive Neuroimaging Unit
CNRS INSERM CEA

Experimental psychologists require sub-millisecond accuracy in measuring stimulus presentation and participant response times. The Black Box ToolKit (BBTK) is a widely used hardware solution for such measurements. One limitation is that the software provided with the BBTK by the vendor is specific to Windows. We developed a cross-platform suite of command line tools and a library designed to interface with the BBTK v3. These tools include serial port detection, sensor threshold adjustment and automated event capture. Additionally, it includes an interactive shell for manual device control and a statistical analysis tool that computes event durations, jitter, and latencies from captured data. Built in Go, the suite is distributed as self-contained binaries for Windows, macOS, and Linux. The suite is documented at <https://chrplr.github.io/bbtkv3> and its source code, and a library for embedding BBTK control in Go applications, are available under the GPL-3.0 licence at <https://github.com/chrplr/bbtkv3>.

Keywords: timing accuracy, stimulus presentation, response time, Black Box ToolKit, open-source software, behavioral research methods

Introduction

Precise timing is a cornerstone of experimental psychology. Variations in stimulus presentation or response recording can introduce noise or even systematic biases that confound experimental results (Plant et al., 2004). While modern computers are capable of high performance, their multi-tasking operating systems and complex hardware architectures (e.g., display buffering, USB polling) often introduce unpredictable latencies.

The Black Box ToolKit (BBTK) was developed to address these challenges by providing an external hardware reference for timing measurements. It acts as a digital oscilloscope and signal generator, capable of capturing activity on light and sound sensors, as well as TTL signals, with sub-millisecond precision. The typical experimental setup involves three components: a) A *stimulation device* (e.g., a computer) running the experimental paradigm; b) The *BBTK hardware* with input sensors (photodiodes, microphones) and TTL inputs/outputs connected to the stimulation device; c) A *host computer* (which may be the same as the stimulation device with our software) driving the BBTK and recording the data (see Figure 1).

The BBTK communicates with the host computer via a serial protocol described in the BBTK API guide (Plant, 2016).



Figure 1

Principle of operation: The BBTK acts as an independent observer of the stimulation device's outputs. An optical sensor detects when the illumination of the screen reaches a certain threshold. Here, the `bbtk-capture` program that drives the BBTK device during events capture runs on the stimulation computer.

The `bbtkv3` software suite that we developed encapsulates this protocol into a set of command-line tools and a high-level library. Designed to be cross-platform (while the vendor's software is specific to Windows), it is easy to install, and can be integrated into automated testing workflows. One of its advantages is that the event capture can run simultaneously on the same computer that runs the stimulation.

 Christophe Pallier Correspondence should be addressed to C. Pallier, INSERM, Neurospin, Gif-sur-Yvette, F-91191, France. E-mail: christophe@pallier.org

```

$ bbtb-set-thresholds 80,0,0,0,110,0,0,0
Trying to open /dev/serial/by-id/usb-BBTK_BBTK_BBTK_V3_BBTKBBTKV3-if00-port0 at
115200 bps...
Getting current thresholds...
{Mic1:63 Mic2:63 Sounder1:63 Sounder2:63 Opto1:63 Opto2:63 Opto3:63 Opto4:63}
Setting new thresholds...:
{Mic1:80 Mic2:0 Sounder1:0 Sounder2:0 Opto1:110 Opto2:0 Opto3:0 Opto4:0}

$ bbtb-capture -d 60 session1
Trying to open /dev/serial/by-id/usb-BBTK_BBTK_BBTK_V3_BBTKBBTKV3-if00-port0 at
115200 bps...
Setting smoothing mask (-s 1,1,0,0,1,1)...
{Mic1:true Mic2:true Opto4:false Opto3:false Opto2:true Opto1:true}
Getting thresholds...
{Mic1:80 Mic2:0 Sounder1:0 Sounder2:0 Opto1:110 Opto2:0 Opto3:0 Opto4:0}
Clearing Timing data... Ok
Capturing events (with DSCM) for 60 seconds...
BBTK-CAPTURE-READY duration=60
[...]
Raw Data saved to session1-001.dat
DSC Events saved to session1-001-dscevents.csv
Events saved to session1-001-events.csv

$ events-stats session1-001-events.csv
[...]
```

Listing 1: Example a recording session. First thresholds are sets, then a capture happens for 60s (option `-d 60`), and finally, a html report is produced with `events-stats`

Core Functionality

Listing 1 provides an example of a recording session. Three commands are used: `bbtk-set-threshold`, `bbtk-capture` and `events-stats`

The recording session starts by calibrating the sensor, setting their sensitivity thresholds at value adequate to distinguish stimuli from background noise or ambient light. The user can use the `bbtk-adjust-thresholds` command to open the interactive adjustment procedure on the Bbtk. Alternatively, the `bbtk-get-thresholds` and `bbtk-set-thresholds` allow him to directly modify the sensitivity thresholds from the command line. The smoothing parameters can also be set directly with `bbtk-set-smoothing`.

Then the `bbtk-capture` is launched. For a specific duration, it records all transitions on the 12 input lines (4 optical, 2 microphone, 2 TTL, and 4 keypad inputs). The transitions, and the associated events are saved in ascii files which can be imported in a spreadsheet, a Python or R notebook for inspection.

To facilitate quick preview of the results, we provide a command-line tool, `events-stats`, which immediately outputs descriptive statistics and ASCII histograms about the timing data, and also writes a Markdown and an HTML report containing the same tables together with log-scale his-

tograms and timeline plots as PNG figures. It computes a) **event durations**, that is, the distribution of how long each sensor remained active; b) **inter-onset intervals**, the timing regularity of successive events on a given sensor; c) **paired-event onset differences**, the latency between a reference event (e.g., TTL trigger) and a target event (e.g., photodiode detection). Listing 2 shows an example of the output from a simple stimulation program (1 frame on — 2 frames off) run on a Raspberry Pi 4.

Selecting the serial port

None of the commands in Listing 1 states which serial port the BBTK is attached to. Every tool that talks to the device resolves the port by itself, in three steps: a) the `-p` option, when one is given on the command line; b) the environment variable `BBTK_PORT`; c) autodetection.

Autodetection first looks for the udev symlink `/dev/serial/by-id/*BBTK*` — the name printed in Listing 1. Linux derives it from the USB descriptors of the device, so, unlike `/dev/ttyUSBn`, which is assigned in enumeration order, it survives replugging and power cycling; the numbering otherwise changes precisely when one has just restarted an unresponsive device. As no such directory exists on macOS and Windows, the tools then fall back on scanning: every available serial port is opened and sent the

```
=== Duration Statistics (ms) ===
```

Type	N	Min	P5	P25	P50	P75	P95	Max	Range	P95-P05	Mean	SD
Opto1	1000	36.5	37.0	37.0	37.2	37.2	37.5	37.8	1.2	0.5	37.2	0.20

```
Opto1:
```

```
histogram (10 bins):
```

```
[ 36.500, 36.625) ms : 2
[ 36.625, 36.750) ms : 0
[ 36.750, 36.875) ms : 28 **
[ 36.875, 37.000) ms : 0
[ 37.000, 37.125) ms : 283 *****
[ 37.125, 37.250) ms : 0
[ 37.250, 37.375) ms : 449 *****
[ 37.375, 37.500) ms : 0
[ 37.500, 37.625) ms : 229 *****
[ 37.625, 37.750) ms : 9
```

```
=== Inter-Onset Interval / Jitter Statistics (ms) ===
```

Type	N	Min	P5	P25	P50	P75	P95	Max	Range	P95-P05	Mean	SD
Opto1	999	49.2	49.8	49.8	50.0	50.0	50.2	50.5	1.2	0.5	50.0	0.19

```
Opto1:
```

```
histogram (10 bins):
```

```
[ 49.250, 49.375) ms : 1
[ 49.375, 49.500) ms : 0
[ 49.500, 49.625) ms : 13 *
[ 49.625, 49.750) ms : 0
[ 49.750, 49.875) ms : 270 *****
[ 49.875, 50.000) ms : 0
[ 50.000, 50.125) ms : 481 *****
[ 50.125, 50.250) ms : 0
[ 50.250, 50.375) ms : 227 *****
[ 50.375, 50.500) ms : 7
```

```
=== Paired-Event Onset Differences relative to TTLin1 (ms) ===
```

Type	N	Min	P5	P25	P50	P75	P95	Max	Range	P95-P05
TTLin1→Opto1	1000	31.0	39.0	39.0	39.2	39.2	39.5	39.8	8.8	0.5

```
TTLin1→Opto1:
```

```
histogram (10 bins):
```

```
[ 31.000, 31.875) ms : 1
[ 31.875, 32.750) ms : 0
[ 32.750, 33.625) ms : 0
[ 33.625, 34.500) ms : 0
[ 34.500, 35.375) ms : 0
[ 35.375, 36.250) ms : 0
[ 36.250, 37.125) ms : 0
[ 37.125, 38.000) ms : 0
[ 38.000, 38.875) ms : 22
[ 38.875, 39.750) ms : 977 *****
```

Listing 2: Print out of `events-stats` from a capture taken while the stimulation computer presented 1000 cycles of 1 frame-on, 2 frames-off. The display monitor refresh rate was fixed at 60Hz. The first section show the detected duration of the on-frames; note that the BBTK adds 20ms to the duration measures because smoothing was set on Opto1. The second section reports on the SOA between successive on-frames. The last section display statistics about the delay between Opto1 activation and a TTL signal that was sent just after the call to display flip. Note the 2-frames delay.

CONN command, and the port answering BBTK; is retained. Scanning therefore writes to all serial ports of the host, and may disturb other instruments attached to it; exporting BBTK_PORT once per session avoids this. The value to use can be obtained from `bbtk-detect-port`, which performs the same scan on demand and reports the ports at which a BBTK answered.

Synchronizing the capture and the stimulation program

When the same computer is running `bbtk-capture` and the stimulation program, it is possible to synchronize both programs.

One solution is to let `bbtk-capture` start the stimulus itself: everything following a `--` separator on its command line is taken as a child process, launched when the device begins to record, as in the Listing below:

```
# ./runexp starts the stimulation
bbtk-capture -d 120 session1 -- ./runexp
```

When the stimulus cannot be a child process, `bbtk-capture` prints the line `BBTK-CAPTURE-READY duration=N` on its standard output, on a line of its own, at the instant recording starts. This can be intercepted by a script controlling the stimulation software.

At the end of the event capture, data are downloaded from the BBTK onto the host PC and saved for further analyses. Among the saved files, a `.dat` file contains the raw output of the BBTK and an `-events.csv` file consists of a table with four columns: `type`, `onset`, `duration`, `durationCorrected`, where `type` identifies the sensor (`Opto1`, `Mic1`, `TTLin1`, ...), and the remaining columns are expressed in milliseconds relative to the start of the capture. `duration` is the value reported by the device, left unaltered; `durationCorrected` has the smoothing tail (see the Limitations section) subtracted on those channels for which smoothing was actually enabled during the capture, and repeats `duration` elsewhere. Onsets need no such correction, since smoothing extends the trailing edge of a detection rather than delaying its onset.

Software Suite Components

All the CLI tools provided by `bbtkv3` are listed Table 1.

These tools build upon the Go module `github.com/chrplr/bbtkv3` which can be imported by developers who want to integrate BBTK control directly into their own Go-based experimental or analysis software, e.g., (Pallier, 2026).

Implementation and Availability

`bbtkv3` is written in Go, leveraging the `go.bug.st/serial` library for cross-platform serial

communication. By using Go, we are able to provide self-contained, statically linked binaries that require no runtime environment (like Python or Java) nor external DLLs.

The software is available for Windows, macOS, and Linux, and Intel and arm architectures for each operating system. Source code and pre-compiled binaries are hosted on GitHub (<https://github.com/chrplr/bbtkv3>) under the GNU General Public Licence v3. Each release is archived on Zenodo: the version described here is v1.0.13 (<https://doi.org/10.5281/zenodo.21620779>), while <https://doi.org/10.5281/zenodo.19551604> always resolves to the most recent release.

Author Contributions

C.P. conceptualized the software, designed the library, and implemented the tools. Claude Sonnet and Gemini 2.5 were used in the final phase of the project, to clean some bugs and to develop the event-stats tool. Consistent with current editorial norms (Springer Nature, 2023), C.P. takes full responsibility for the content of the work.

References

- Pallier, C. (2026). Goxpyriment: A high-level Go framework for building behavioral and psychological experiments (github repo: <https://chrplr.github.io/goxpyriment>) [Accessed: 2026-04-01].
- Plant, R. (2016). *The black box toolkit v2. api guide. revision rc4*. The Black Box ToolKit Ltd.
- Plant, R., Hammond, N., & Turner, G. (2004). Self-validating presentation and response timing in cognitive paradigms: How and why? *Behavior Research Methods, Instruments, & Computers*, 36(2), 291–303. <https://doi.org/10.3758/BF03195575>
- Springer Nature. (2023). Authorship and ai tools. <https://www.springernature.com/gp/editorial-policies-authorship>

Table 1*The bbtkv3 toolset*

Tool	Description
bbtk-detect-port	Scans serial ports to locate the connected BBTK.
bbtk-capture	Captures events for a specified duration and exports them to CSV and binary formats.
bbtk-input-check	Provides a live stream of sensor states for real-time validation.
bbtk-adjust-thresholds	Interactive menu for sensor sensitivity calibration.
bbtk-get-thresholds	Reads current sensor thresholds from the device.
bbtk-set-thresholds	Writes new threshold values to the device.
bbtk-set-smoothing	Enables or disables smoothing on individual sensor channels.
bbtk-event-marking	Configures the device for real-time event marking.
bbtk-trigger-response	Programs the device to answer a trigger on an input line with a delayed pulse on an output line, timed entirely in firmware.
bbtk-send-command	Sends raw protocol commands read from standard input and prints the device's replies.
bbtk-send-break	Sends the break character to interrupt a running device operation and recover a device left streaming.
get-serial-port-list	Lists the serial ports available on the host.
events-stats	Performs statistical analysis on captured timing data.
ibbtk	An interactive shell providing access to all device functions.